

Vectrace Emb CRM - Project Overview

Prepared: 3 September 2026

Evidence: source code, package configuration, and supabase/schema.sql in this repository.

Secrets: intentionally redacted. Environment variable names are listed, never values.

1. What this project is

Vectrace Emb CRM is an internal, desktop-first CRM for an embroidery/digitizing/vector-art/custom-patches business. It is intended for the business team-not for customers directly.

The CRM tracks a lead from initial contact through a five-stage opportunity process, free-trial work, conversion to a client, project assignment to a designer, delivered files, payments, expenses, reports, Gmail-based email lookup, and notifications.

Current user types

| User type | What the code permits them to see and do |

| --- | --- |

| Admin | Full dashboard; all leads; import and bulk-assign leads; clients; designers; project assignments; designer payments; finance; reports; users; create/rename/role-assign CRM accounts; delete leads/clients/designers; upload client/trial files; assign work and send designer email. |

| Caller | Their own assigned/captured leads; add/edit leads; call logs; opportunity stages; POCs; follow-ups; trials; Gmail matching-email lookup; upload trial files. They cannot access Clients, Designers, Finance, Reports, Users, or delete leads. |

| Designer | Only My Work. They see their own assignments, can download authorized client files, change status, upload/submit final files, and remove their own delivered files. Sensitive client-company and designer-email fields are omitted from assignment responses sent to designers. |

The browser navigation matches this: Admin sees Dashboard, Leads, Opportunities, Clients, Designers, Finance, Reports, and Users; Caller sees Dashboard, Leads, and Opportunities; Designer sees only My Work.

2. What actually works

This is based on implemented routes and UI code, not on the original README. The README still calls several functions "deferred" even though parts of them have since been implemented.

Working in code (subject to live-service credentials being valid)

- Email/password sign-in through Supabase Auth, with roles from `crm_profiles`.
- Admin invitation flow: the CRM can invite a user, create/update their CRM role, and let the invitee choose a password.
- 14-day server sessions, stored as SHA-256 hashes in the sessions CRM collection. The raw session token is stored in browser local storage.
- Leads: create, list/filter, import, edit, delete (Admin), assign/bulk assign, follow-ups, manual call logs, statuses, and duplicate checks by phone/company.
- Opportunity workflow: POCs, Stage 3 email log, free-trial details and attachments, conversion/ordering. The computed stages are: 1 call connected, 2 POC, 3 email sent, 4 free trial, 5 paid/won.
- Clients, client orders, payments, finance summaries, expenses, and basic sales/caller/finance reports.
- Designer directory, assignment, project status, designer payment tracking, up to six delivered files per assignment, and Admin download of delivered files.
- Cloudflare R2 file upload/download through the backend. Uploads are limited to 25 MB and a defined extension allow-list.
- File authorization: Admin can download all authorized CRM files; Callers only trial files on leads they own/captured; Designers only files tied to their own assignment.
- Gmail OAuth for Admin/Caller accounts. It can load up to three recent matching emails for a selected POC and can send a plain-text project-assignment email from a connected Admin Gmail account.
- Persistent in-app notifications and optional Web Push subscription/notifications. Browser push requires the user to click Enable alerts and grant browser permission.
- Automated tests exist and previously passed locally: 16 backend tests. Unverified at the time of this report: current deployed test status, because no deployed environment is defined in the repository.

Half-built / incomplete / fragile

- Hosting is not configured in source. There is no `render.yaml`, Cloudflare Worker/Pages configuration, Dockerfile, CI workflow, production URL, or deployment migration. The project is definitely runnable locally; an online deployment is unverified.
- The frontend supports `VITE_API_URL` for a deployed API, but the repository does not provide a production environment template or hosting setup.
- Gmail connection state (state parameter) is held in a process-local Map. It works on one continuously running backend process, but can fail if the server restarts between clicking Connect Gmail and Google returning to the callback.
- Notifications are persisted, but delivery is best-effort: notification writes and browser push are fired asynchronously rather than inside a transaction/job queue.
- The browser notification service worker exists, but the project has no deployed HTTPS site. Push behaviour in a real production browser is therefore unverified.
- The finance model is operational but not accounting-grade: it defaults to a manual USD->PKR rate of

278; currencies and totals are partly embedded inside JSON records; no audit trail, invoice, tax, reconciliation, approval flow, or immutable ledger exists.

- There is no formal API specification, versioning, rate limiting, request validation library, audit log, monitoring, backups policy, or automated deployment.
- Data records are JSON documents. This is flexible while the CRM changes, but makes joins, reporting, database constraints, and a customer portal harder than with normalized tables.
- Data can become duplicated: assignment/project data is copied into client projects, and delivered paths are copied into both assignments and projects. The application attempts to keep them in sync, but the database cannot enforce it.
- Deleting a lead/client/designer removes CRM records, but R2 object deletion is not implemented. Uploaded file objects can become orphaned.
- readme demo credentials and "deferred" statements are outdated. The fallback in-memory demo credentials are still compiled into the backend and should be removed or guarded before a public production deployment.

Planned only / not implemented

- Customer-facing portal with customer authentication, customer-scoped orders/projects/files/invoices, or self-service database access.
- Zoom Phone click-to-dial and automatic Stage 1 call logging. No Zoom SDK, API client, routes, webhooks, tables, or environment variables exist in this repository.
- Automatic call qualification rule requested by the business (connected for at least one minute, excluding voicemail). Current manual call logging accepts any duration above zero; caller reporting counts calls at 30 seconds or more.
- CRM email sending for arbitrary emails/Stage 3. Current Gmail code only sends designer assignment emails; Stage 3 loads matching Gmail messages and saves a selected email log.
- Email attachment ingestion into the CRM. Gmail reader returns message subject/body metadata, not attachments.
- True real-time notifications/websockets; current UI polls notifications every 10 seconds.
- Payment gateway, live FX rates, custom pipeline stages, audit timeline, global search across every entity, external customer API, Zoom integration, or sync jobs.

3. Stack and runtime

Layer	Implemented technology
Frontend	React + TypeScript + Vite; Lucide icons; CSS files.
Backend	Node.js + TypeScript + Fastify.
Data access	Hand-written REST calls from backend to Supabase PostgREST/Auth; repository pattern. No Supabase JavaScript SDK.
Authentication	Supabase Auth email/password and invitations; backend-created CRM session token.
Object storage	Cloudflare R2, accessed with AWS S3 SDK-compatible API.
Email	Google OAuth 2.0 and Gmail REST API using native fetch.
Browser alerts	Web Push (web-push) and a service worker at frontend/public/push-sw.js.
Local development	Vite at port 5173, proxying /api to Fastify at port 3001.
Hosting	Unverified / not configured in repository. The source can run locally. It has no checked-in production host configuration.

Required secret configuration is kept outside source in .env: SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, SUPABASE_SERVICE_ROLE_KEY, R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET_NAME, R2_ENDPOINT, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_GMAIL_REDIRECT_URI, PUSH_VAPID_PUBLIC_KEY, PUSH_VAPID_PRIVATE_KEY, and PUSH_CONTACT_EMAIL. The deployed frontend additionally expects VITE_API_URL when it is not served behind the local /api proxy.

4. Cloudflare and Supabase

What lives where

Service	What the code uses it for
Supabase	PostgreSQL tables crm_records and crm_profiles; Supabase Auth users; role profiles; all CRM JSON records including leads, clients, assignments, expenses, notifications, sessions, push subscriptions, and encrypted Gmail connections.
Cloudflare R2	Binary files only: client/trial attachments and designer-delivered files. Objects are saved under crm-uploads/<UUID>.
Cloudflare Pages / Workers / D1 / KV	Not used in the repository. No source/configuration demonstrates that any of these Cloudflare products are deployed.

Why both?

They serve different purposes: Supabase provides relational Postgres and authentication; R2 provides cheaper object storage for binary artwork/files. The backend stores a file reference such as /files/<UUID>/<encoded name> in a Supabase JSON record, while the bytes are stored in R2 under crm-uploads/<UUID>.

This looks like a deliberate practical separation for database/auth versus files. However, there is no

architectural decision record, migration history, or infrastructure configuration in the repository, so the original decision rationale is unverified.

How they stay in sync

They do not have database-level synchronization. The Fastify backend is the coordinator:

1. It uploads the bytes to R2.
2. It saves the generated file path into the relevant JSON record in Supabase.
3. On download, it authorizes the current CRM user using Supabase-backed records, then fetches the bytes from R2.

This has two important consequences:

- If R2 upload succeeds but the Supabase write fails, an orphan R2 object is possible.
- Deleting CRM records does not delete R2 objects. Cleanup/lifecycle rules are not implemented in source.

5. Database schema and migrations

There is no migrations directory or migration runner. The only database definition is `supabase/schema.sql`; it must be pasted/run manually in Supabase SQL Editor. Below is the complete SQL exactly as checked in:

```
-- Run this once in Supabase: SQL Editor -> New query -> paste -> Run.
-- CRM objects are stored as version-friendly JSON records while the CRM is evolving.
create table if not exists public.crm_records (
  collection text not null,
  id text not null,
  data jsonb not null,
  created_at timestamptz not null default timezone('utc', now()),
  updated_at timestamptz not null default timezone('utc', now()),
  primary key (collection, id)
);

create index if not exists crm_records_collection_updated_at_idx
on public.crm_records (collection, updated_at);

alter table public.crm_records enable row level security;

-- Browser users get no direct table access. The protected CRM backend uses the secret key.
revoke all on public.crm_records from anon, authenticated;
grant select, insert, update, delete on public.crm_records to service_role;

-- Real CRM login roles. Each row is linked to exactly one Supabase Auth user.
create table if not exists public.crm_profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  role text not null check (role in ('Admin', 'Caller', 'Designer')),
  designer_id text,
  created_at timestamptz not null default timezone('utc', now())
);

alter table public.crm_profiles enable row level security;
revoke all on public.crm_profiles from anon, authenticated;
grant select, insert, update, delete on public.crm_profiles to service_role;
```

Tables, columns, and relationships

public.crm_records

Column	Type	Meaning
--------	------	---------

---	---	---
-----	-----	-----

collection	text, not null	Logical collection name. Current code uses: leads, clients, assignments, designers, expenses, users, notifications, sessions, push_subscriptions, and gmail_connections.
------------	----------------	--

id	text, not null	Object ID within the collection. Together with collection this is the primary key.
----	----------------	--

data	jsonb, not null	Entire application object. This is where all business fields live.
------	-----------------	--

created_at	timestamptz, not null	Defaults to UTC now.
------------	-----------------------	----------------------

updated_at	timestamptz, not null	Defaults to UTC now, but there is no trigger in the migration to update it automatically on PATCH.
------------	-----------------------	--

Relationship enforcement: none inside this table. Links such as Lead -> Client, Client -> Project, Assignment -> Designer, and files are IDs/names/paths inside JSON, enforced by application code only.

public.crm_profiles

Column	Type	Meaning
--------	------	---------

---	---	---
-----	-----	-----

id	uuid, primary key	Required foreign key to auth.users(id); cascades on Auth-user deletion.
----	-------------------	---

full_name	text, not null	Display name used for the CRM role and notifications.
-----------	----------------	---

role	text, not null	Database check constraint: Admin, Caller, or Designer.
------	----------------	--

designer_id	text, nullable	Application link to a designers JSON object for a Designer user. No foreign key.
-------------	----------------	--

created_at	timestamptz, not null	Defaults to UTC now.
------------	-----------------------	----------------------

JSON record shapes used by application code

The following are not SQL tables; they are TypeScript object shapes stored in `crm_records.data`.

- Lead: company/contact/phone/address details, lead status, owner/capturedBy/assignment metadata, calls, follow-ups, and optional nested opportunity.
- Opportunity (inside Lead): POCs, emailSent, free-trial details/attachments, and orders.
- Client: company/contact data, paid/unpaid state, projects array, payments array, totals, and optional convertedFrom lead ID.
- Project (inside Client): project name/service/designer/status/client attachment paths/designer attachment paths.
- Assignment: designer and client/project details, payment/paid fields, attached file paths, status, deliveries, and delivery note.
- Designer: name, email, optional address/bank account/phone.
- Expense: name, amount, date, category, description, currency/original/conversion fields.
- Notification: recipient name, message, kind, created time, read flag.
- Session: SHA-256 token hash as id, user identity/role, optional designer ID, expiry. The raw token is not stored in Supabase.
- Push subscription: user ID, browser endpoint and public key material.
- Gmail connection: user ID/email/connected timestamp plus an encrypted refresh token. Encryption derives from SUPABASE_SERVICE_ROLE_KEY; rotating that key without a migration would make existing stored Gmail tokens unreadable.

Row-level security (RLS)

RLS is enabled on both application tables. There are no per-user USING/WITH CHECK policies. Instead, direct access is revoked from anon and authenticated; only Supabase service_role is granted CRUD rights. Therefore all authorization is enforced in the backend application. This is safe only while the service-role key remains private and the backend authorization stays correct.

6. Authentication and authorization

- Sign-in provider: Supabase Auth email/password. No Google/Microsoft/social sign-in provider is implemented for CRM login.
- Account creation: Admin can invite a user from CRM, which calls Supabase Auth's invite endpoint; the recipient sets their own password through the invitation/recovery token screen.
- Roles: stored in crm_profiles, then loaded at login. The backend rejects a valid Supabase Auth login if no CRM profile exists.
- Sessions: after login, Fastify creates its own random session token. The browser sends it as Authorization: Bearer <token>. The backend stores only SHA-256 token hashes in crm_records for 14 days, restores them at startup, and removes expired entries.
- Role enforcement: routes call a central allow(session, roles) helper, plus record ownership checks for callers/designers. The frontend hides navigation items, but security is meant to come from backend checks.
- CORS: configured as origin: true, meaning the backend reflects/permits any origin. This is convenient for development but should be narrowed to the real CRM site before production.

7. File storage and access control

Uploads are accepted by the backend, not by the browser directly to R2. Accepted types are .ai, .dst, .emb, .eps, .jpeg, .jpg, .pdf, .pes, .png, .svg, .webp, and .zip; maximum size is 25 MB.

Files are saved in R2 as crm-uploads/<UUID>. Supabase only stores a CRM route/path reference, such as /files/<UUID>/design.png.

Download access is checked before the backend fetches R2 bytes:

- Admin: any CRM-linked file.
- Caller: trial attachments for leads they own or captured.
- Designer: files on assignments where designerEmail matches their session email.

The frontend does not receive public R2 URLs; it downloads through the authorized backend endpoint. R2 object deletion/lifecycle cleanup is absent.

8. API surface

The project has an internal JSON HTTP API under /api; it is not documented, versioned, rate-limited, or intended as a public customer/developer API. All endpoints except /health and the Google OAuth callback require a backend CRM session.

Major groups:

- /api/auth/* - login, logout, invite password completion.
- /api/gmail/* - status, OAuth connect/callback, matching POC emails.
- /api/notifications/* and /api/push/* - in-app and browser notification support.
- /api/files - upload and authorized download.
- /api/leads/*, /api/opportunities - lead/opportunity/call/order/follow-up work.
- /api/clients/*, /api/designers/*, /api/assignments/* - operational client/project/designer work.
- /api/finance/summary, /api/reports/*, /api/expenses - finance/reporting.
- /api/users/* - Admin account/role/name management.

For a future client portal, do not call Supabase tables directly and do not expose

SUPABASE_SERVICE_ROLE_KEY in that portal. Create a small, documented, versioned portal API or a dedicated backend-for-frontend layer instead.

9. Known problems and warnings for a new developer

1. No production deployment is tracked. Do not assume Render/Cloudflare hosting exists because it was discussed elsewhere. Verify public URLs, HTTPS, DNS, Google OAuth redirect URI, environment variables, R2 bucket policy, and push keys before launch.
2. One generic JSON table is a temporary MVP design. It has no foreign keys/unique constraints for business entities, no indexed fields inside JSON, and weak data integrity for a growing business.
3. RLS delegates all security to one backend. Never give the frontend the service-role key. Add API tests for every authorization boundary before opening a customer portal.
4. `updated_at` does not automatically update. The SQL declares it but has no trigger, while the backend PATCH updates data only. Ordering by it may be stale.
5. Gmail OAuth is process-state dependent during connection. Restarting/redeploying during OAuth can lose the state value. Google OAuth consent/testing/production verification status is not shown in this repository.
6. Gmail refresh-token encryption is coupled to the Supabase service role key. Key rotation needs a planned re-encryption/reconnect procedure.
7. No durable background jobs. Gmail sends, push sends, notification persistence, and future webhooks have no retry/dead-letter queue.
8. No R2 cleanup. Deletes can leak uploaded files/storage cost and potentially leave sensitive files in storage.
9. No audit log or soft-delete. Admin deletions are destructive at the CRM record level; there is no "who changed what" history.
10. No schema validation at the HTTP boundary. Requests are mostly TypeScript casts at runtime. A malformed client/API request could create inconsistent JSON.
11. Hard-coded fallback demo accounts exist in code. They are used only if Supabase Auth is not configured, but should be disabled for any public production build.
12. Role/name matching is brittle in places. Some notification/lead ownership logic uses display names or first names instead of stable user IDs; duplicate/renamed names can cause incorrect targeting.
13. No tests cover integrations or authorization end-to-end. Existing tests are backend unit-level; run an end-to-end test suite before launch.

10. Customer portal integration recommendation

Business goal

The separate customer portal should let each customer see only their own company profile, orders/projects, approved files/deliveries, invoices/payments, and perhaps submit a new order or artwork. It must never expose internal leads, caller notes, designer bank details, other customers, or the Supabase service-role key.

Honest recommendation: normalize the database first, then use an API

Do not connect the portal directly to the current `crm_records` JSON table. Direct shared-table access would force portal code to understand internal CRM document shapes and makes customer-scoped RLS difficult. It also risks exposing all CRM data when a policy is wrong.

Recommended design:

1. Keep Supabase as the shared database and Auth provider.
2. Add normalized portal-ready tables: `organizations`, `customer_users`, `clients`, `projects`, `project_files`, `orders`, `invoices`, `payments`, and optionally `portal_events`/`audit_events`.
3. Give every client and project stable UUIDs. Link each portal user to exactly one organization/client (or a controlled many-to-many access table).
4. Add proper RLS policies so a customer can only read/write rows belonging to their organization. Keep Admin/Caller/Designer internal roles separate from customer roles.
5. Put business actions behind a documented backend API (for example `/api/v1/portal/*`) or Supabase Edge Functions. Use signed, short-lived file downloads rather than public R2 URLs.
6. Migrate existing CRM JSON records to the normalized tables through a tested, idempotent migration. Until migration is complete, treat the CRM backend as the only writer to existing records.

Short-term bridge (if portal work must start now)

Build a separate portal API against the existing CRM backend. Add narrowly scoped read endpoints such as `GET /api/v1/portal/me/projects` and `GET /api/v1/portal/projects/:id/files`, backed by an explicit customer-user-to-client mapping. Do not expose generic `/api/clients` or `/api/files` to portal users. This bridge is safer than direct table access, but it is still temporary because the underlying JSON model does not scale cleanly.

Why not "shared database only"?

Shared Supabase is fine as infrastructure, but the portal needs a different security boundary. The internal CRM has broad staff access and uses service-role backend access. The customer portal needs hard organization isolation at database and API levels. Normalized tables plus RLS and a portal-specific API offer the clearest, safest long-term model.

Conclusion

The project is a functioning internal CRM MVP with Supabase Auth/data, R2 file storage, Gmail integration, and browser notification support in source. It is not yet a production-ready platform or customer portal. The highest-value next technical work is: deploy and secure the backend; remove/guard demo mode; add observability/backups/audit logging; normalize client/project/order/file data; then build the customer portal against a dedicated customer-safe API and RLS model.